

CYB 4800

Senior Capstone Project

Spring 2026

Final Report

for

Design and Development of Fake Job Posting Detection Tool

Jynalis Diaz

Faculty Instructor

Dr. Sagar Raina

Table of Contents

3.1 Introduction	3
3.2 Objectives	5
3.3 Problem Specification	6
3.4 Solution Design	8
3.4 a. Project File/ Directory Structures	8
3.4b System Architecture Overview	23
3.4c Machine Learning Algorithms	23
3.4d Implementation	24
3.5 Results	25
3.6 Conclusion	28

3.1 Introduction

Fake job listings are a major issue, especially among students looking for internships and entry-level jobs. In order to get personal information from candidates who might not be aware of the warning signs, such as resumes, phone numbers, and even Social Security numbers, scammers create fake postings. Having personally experienced this, I wanted to create something that can help others avoid a similar mistake.

This project is a web-based program called the Fake Job Posting Detection Tool, which allows students to paste in a job posting and receive an analysis on whether it appears authentic or fraudulent. It differs from comparable tools in that it does more than simply render a decision. It emphasizes the various red flags found in the posting and explains in plain language why each one could be a concern, giving the user more knowledge rather than just a warning.

The web interface was built using HTML and CSS, the backend was developed with Python and Flask on a Kali Linux virtual machine, and the history of previous scans is stored in a SQLite database. Classification is handled by a machine learning model trained on the Kaggle EMSCAD dataset using Logistic Regression with TF-IDF, while a separate rule-based detection layer identifies specific warning patterns such as cryptocurrency mentions, suspicious email addresses, requests for personal information, and unusual contact methods like Telegram or WhatsApp. Together, these two methods catch what the other might miss.

This project's primary purpose was to build on an existing open-source tool so that students could genuinely learn from it, not just receive a verdict. The focus was on improving detection accuracy, adding meaningful explanations for each flag, and creating an experience that gradually builds awareness over time.

3.2 Objectives

The main objective of this project is to improve an existing fraudulent job posting detection program so that students seeking internships and remote employment can identify them more easily. The updated program not only classifies posts as legitimate or fake, but also identifies and explains warning flags, making it both a detector and an instructional tool. To do this, I examined common scam traits such as confusing job descriptions, early demands for personal information, unverified employer data, and unusual contact techniques. On top of the present detection engine, these indicators will be used to develop an explanation layer that communicates results in a way that students can genuinely understand and learn from.

The ultimate objective is to teach students how to spot suspicious postings on their own over time, rather than relying solely on a program to provide an answer. Through this project, I hope to improve my abilities in altering existing systems, working with text analysis, and completing an entire project lifecycle, from research and design to implementation, testing, and documentation.

3.3 Problem Specification

Many students searching for jobs and internships, especially remote positions, come across fake job postings. These postings are created by scammers pretending to be real employers to collect personal information such as email addresses, phone numbers, and resumes. Once shared, this information can be misused or lead to identity theft.

Fake job postings can look very convincing and may appear on popular job platforms, but they often contain warning signs. These can include vague job descriptions, early requests for personal information, missing company details, or unusual contact methods, such as WhatsApp messages. Students may not always recognize these signs, especially when eager to secure a job. As a student myself, I have experienced this difficulty firsthand, which motivated this project.

Various approaches have been used to address this issue. Pillai (2023) proposed a high-accuracy Bidirectional LSTM model for detecting fraudulent job advertising by analyzing textual and numerical characteristics in job listing data. Community-developed technologies, such as the fake Job Detector projects on GitHub and Devpost, allow anyone to post job vacancies and receive a true or false classification. Despite demonstrating that automated detection is possible, all of these devices have the same disadvantage: they offer a shallow result without context. They decide without explaining whether certain features of the advertisement are dubious or why they are viewed as red flags.

The purpose of this project was to create a system that not only helps students identify potentially fake or risky job postings before they apply, but also explains the reasoning behind each flag in plain language. Rather than returning a simple verdict, the tool was able to highlight specific warning signs found within a posting and provide a brief explanation of why each one is

considered suspicious. This approach treats the user as someone who needs to learn, not just someone who needs an answer.

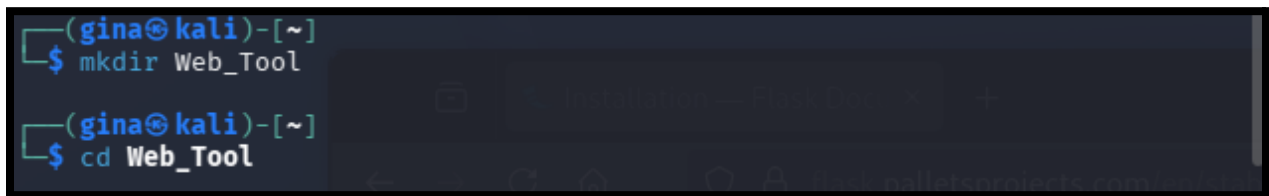
The system still does not fully guarantee that a job is fake, but it will provide meaningful guidance and actively increase a student's ability to recognize risk on their own. Overall, this project aims to help students protect their personal information and feel more confident and informed when applying for jobs online.

3.4 Solution Design

The Fake Job Posting Detection Tool is built across three distinct architectural layers that work together to analyze a submitted job posting and return results to the user. Each layer has a defined responsibility, and together they create a pipeline that takes raw job posting text as input and returns a classification verdict, a fraud confidence score, and a set of detailed flag explanations as output.

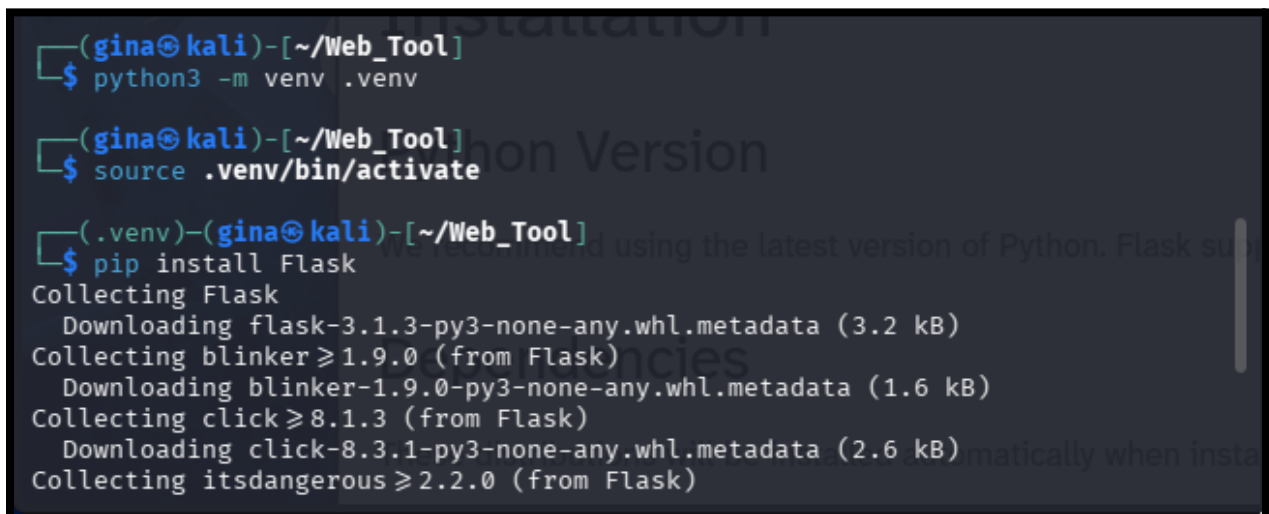
The architecture's modular design allowed for the independent setup, testing, and improvement of each component. This was useful throughout development since it made it easy to upgrade the rule-based detection layer and the machine learning model independently without impacting the database logic or web interface.

3.4 a. Project File/ Directory Structures

A terminal window with a dark background. The prompt is (gina@kali)-[~]. The first command is \$ mkdir Web_Tool. The second command is \$ cd Web_Tool. The prompt changes to (gina@kali)-[~/Web_Tool].

```
(gina@kali)-[~]  
$ mkdir Web_Tool  
  
(gina@kali)-[~]  
$ cd Web_Tool  
(gina@kali)-[~/Web_Tool]
```

(Image above shows a created directory named Web_Tool, which contains files and data of the created application. I then went into the directory to start installing libraries for further development)

A terminal window with a dark background. The prompt is (gina@kali)-[~/Web_Tool]. The first command is \$ python3 -m venv .venv. The second command is \$ source .venv/bin/activate. The prompt changes to (.venv)-(gina@kali)-[~/Web_Tool]. The third command is \$ pip install Flask. The output shows the installation of Flask and its dependencies: blinker, click, and itsdangerous.

```
(gina@kali)-[~/Web_Tool]  
$ python3 -m venv .venv  
  
(gina@kali)-[~/Web_Tool]  
$ source .venv/bin/activate  
(.venv)-(gina@kali)-[~/Web_Tool]  
$ pip install Flask  
Collecting Flask  
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)  
Collecting blinker<=1.9.0 (from Flask)  
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)  
Collecting click<=8.1.3 (from Flask)  
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)  
Collecting itsdangerous<=2.2.0 (from Flask)
```

(This image shows a Python virtual environment created in the directory of 'Web_Tool' to ensure that dependencies would remain isolated, and Flask (a web framework that connects the backend logic to the user-facing interface, handling routing and requests between the two) was installed as the main web framework for the application itself.)

(The main Flask application file loads the trained model, initializes the database, and defines the warning flags and routes that handle user submissions.)

```
GNU nano 8.7.1 APP.py
from flask import Flask, render_template, request, session
from database import init_db, save_analysis, get_all_analyses, get_recent_analyses
import pickle
import re

app = Flask(__name__)
app.secret_key = "fakejobdetector2024"
APP_NAME = "Fake Job Posting Detection Tool"

# Load trained model
with open("model/model.pkl", "rb") as f:
    model = pickle.load(f)

# Initialize database when app starts
init_db()

# Strong flags that can override the model verdict
STRONG_FLAGS = [
    "known scam number detected",
    "suspicious phone number detected",
    "suspicious email address detected",
    "suspicious email format detected",
    "personal email used instead of company email",
    "asks for payment or fees",
    "requests personal info early",
    "cryptocurrency payment",
    "google form or unofficial application"
]

# Warning flags with detailed explanations
WARNING_FLAGS = {
    "requests personal info early": {
        "patterns": [r"\b(ssn|social security|bank account|credit card|passport)\b"],
        "explanation": (
            "Legitimate employers do not ask for sensitive personal information such as your "
            "Social Security Number, bank account details, or passport information during the "
            "application stage. This type of information is only needed after you have been "
            "officially hired and is handled through secure HR systems. If a job posting or "
            "recruiter asks for this upfront, it is a strong indicator that they are attempting "
            "to collect your data for identity theft or financial fraud. Never share this "
            "information before verifying the employer is real."
        )
    },
    "asks for payment or fees": {
        "patterns": [r"\b(upfront fee|registration fee|training fee|deposit required|wire transfer|pay to apply)\b"],
        "explanation": (
            "Real employers pay you – you never pay them. Any job that requires an upfront "
            "payment, deposit, or fee to get started is almost certainly a scam. This is a "
            "classic fraud tactic where scammers convince applicants to send money for things "
            "like training materials, background checks, or equipment, and then disappear."
        )
    }
}
```

```
GNU nano 8.7.1 APP.py
    "Legitimate companies cover their own hiring costs. If money is being requested "
    "from you before you have even started working, walk away immediately."
    )
},
"vague or no company info": {
    "patterns": [r"\b(no experience required|anyone can apply)\b"],
    "explanation": (
        "Phrases like 'no experience required' or 'anyone can apply' are commonly used "
        "in fraudulent postings to cast a wide net and attract as many applicants as "
        "possible. While some entry level jobs do exist, legitimate postings will still "
        "describe specific responsibilities, team structure, and company background. "
        "A real employer wants the right candidate, not just any candidate. Vague postings "
        "with no detail about the company, its location, or its industry are a red flag "
        "that the poster may not represent a real organization."
    )
},
"unusual contact method": {
    "patterns": [r"\b(whatsapp|telegram|text us|direct message)\b"],
    "explanation": (
        "Professional companies communicate through official business email addresses or "
        "phone systems, not personal messaging apps like WhatsApp or Telegram. When a "
        "recruiter insists on communicating through these platforms it is often because "
        "they want to avoid a paper trail and make it harder for you to verify their "
        "identity. Scammers use these apps because accounts are easy to create, hard to "
        "trace, and allow them to impersonate real companies. Always insist on "
        "communicating through official company channels."
    )
},
"unrealistic salary or promise": {
    "patterns": [r"\b(earn \$[0-9,]+ per (day|week)|guaranteed income|risk.?free)\b"],
    "explanation": (
        "Promises of unusually high pay for little work, guaranteed income, or risk-free "
        "earnings are major warning signs. Scammers use these claims to make their postings "
        "attractive and create a sense of urgency so you apply without thinking critically. "
        "In reality, no legitimate job can guarantee income or promise risk-free results. "
        "If the salary being offered seems too good to be true for the type of work being "
        "described, it almost certainly is. Research typical salaries for the role on "
        "sites like Glassdoor or LinkedIn before applying."
    )
},
"cryptocurrency payment": {
    "patterns": [r"\b(bitcoin|crypto|ethereum|usdt|binance|wallet address)\b"],
    "explanation": (
        "Requests for cryptocurrency payments or mentions of crypto wallets in job postings "
        "are a major modern scam indicator. Legitimate employers never pay salaries or "
        "request payments in cryptocurrency because it is untraceable and irreversible. "
        "Scammers use crypto specifically because once you send it, it cannot be recovered. "
        "If a job mentions crypto in any financial context, treat it as a serious red flag."
    )
},
},
```

(When a match is found, the user is sent a plain-language explanation for each flag category that correlates to a repeating pattern.)

```
GNU nano 8.7.1 APP.py
    },
    "google form or unofficial application": {
        "patterns": [r"\b(google form|docs\.google|bit\.ly|tinyurl)\b"],
        "explanation": (
            "Legitimate companies use official application portals on their own websites or "
            "established platforms like LinkedIn, Indeed, or Handshake. Asking applicants to "
            "fill out a Google Form or click a shortened URL is a common scam tactic used to "
            "collect personal information without any accountability. Shortened links can also "
            "lead to phishing websites designed to steal your login credentials."
        )
    },
    "remote only interview red flag": {
        "patterns": [r"\b(zoom interview only|no in.?person|fully remote hiring|hired without interview)\b"],
        "explanation": (
            "While remote interviews are common and legitimate, postings that emphasize you "
            "will be hired without any real interview process or that explicitly avoid any "
            "in person or video verification are suspicious. Scammers avoid face to face "
            "contact because it makes them easier to identify and report. A legitimate employer "
            "will always want to properly vet candidates through a real interview process."
        )
    },
    "work from home unrealistic claims": {
        "patterns": [r"\b(set your own hours|be your own boss|work from anywhere|no boss)\b"],
        "explanation": (
            "Phrases like 'set your own hours', 'be your own boss', or 'work from anywhere' "
            "are hallmarks of multi-level marketing schemes and job scams targeting students. "
            "Real remote jobs still have schedules, managers, and accountability. These phrases "
            "are designed to appeal to people seeking flexibility and independence, but they "
            "rarely represent legitimate employment. Always look for specific job duties, "
            "reporting structures, and company details before applying."
        )
    },
}

# Known scam numbers
KNOWN_SCAM_NUMBERS = [
    "+1 (363) 201-2819",
    "+1(205)651-9396",
]

# Known scam email patterns
KNOWN_SCAM_EMAIL_PATTERNS = [
    r"aide\d+\@w+a",
    r"\w+\d{3,}\@w+a",
    r"@pastryofistanbul\.com",
]

# Fake area codes
FAKE_AREA_CODES = ["363", "372", "353", "382"]
```

(Scans the submitted posting text against all flag patterns, known scam numbers, suspicious area codes, and email checks, then returns a list of any matches found.)

```

        if re.search(pattern, text_lower):
            found.append({
                "flag": "suspicious email address detected",
                "explanation": (
                    "This posting contains an email address that matches known scam patterns. "
                    "Fraudulent recruiters often use personal emails with random numbers, "
                    "unrelated business domains, or the '+' trick to create throwaway addresses "
                    "that are hard to trace. For example an email like "
                    "aiden45285+invst@pastryofistanbul.com uses a personal name, random numbers, "
                    "a '+' variation, and a completely unrelated domain, none of which belong "
                    "to a real employer. Always verify that the email domain matches the "
                    "company's official website before responding."
                )
            })
            break

# Check for personal email domains
personal_email_pattern = r'\b[\w\.\+]+@(\gmail|yahoo|hotmail|outlook|aol)\.com\b'
if re.search(personal_email_pattern, text_lower):
    found.append({
        "flag": "personal email used instead of company email",
        "explanation": (
            "A legitimate company will always contact you from an official business email "
            "address that matches their company domain such as hr@companyname.com. When a "
            "recruiter uses a personal Gmail, Yahoo, or Hotmail address it means either the "
            "company does not exist or the person is not actually affiliated with the company "
            "they claim to represent. This is one of the easiest red flags to spot and one "
            "of the most reliable indicators of a fraudulent posting. Search the company "
            "name independently and find their official contact information to verify."
        )
    })

# Check for + trick in email
plus_email_pattern = r'\b[\w\.\+]+d+([\w\.\+]+)'
if re.search(plus_email_pattern, text_lower):
    found.append({
        "flag": "suspicious email format detected (+ trick used)",
        "explanation": (
            "The '+' symbol in an email address is used to create variations of the same "
            "inbox, for example name+tag@gmail.com still delivers to name@gmail.com. "
            "Scammers exploit this feature to create many throwaway contact addresses that "
            "all funnel into one inbox making them harder to track and block. No legitimate "
            "employer uses this format for professional communication. If you see a '+' in "
            "a recruiter's email address treat it as a serious red flag and do not respond."
        )
    })

return found

@app.route("/", methods=["GET"])

```

(Checks for suspicious email formats including known scam domains, personal email addresses used in place of company email, and the + alias trick.)

```
GNU nano 8.7.1 APP.py
def index():
    if "history" not in session:
        session["history"] = []
    return render_template("index.html", app_name=APP_NAME, history=session["history"])

@app.route("/analyze", methods=["POST"])
def analyze():
    job_text = request.form.get("job_text", "")

    prediction = model.predict([job_text])[0]
    probability = model.predict_proba([job_text])[0][1]
    flags = check_flags(job_text)

    # Only override model if a strong flag is detected
    strong_flag_found = any(
        any(strong in item["flag"] for strong in STRONG_FLAGS)
        for item in flags
    )

    if strong_flag_found and prediction == 0:
        probability = max(probability, 0.75)
        prediction = 1

    result = {
        "verdict": "LIKELY FAKE" if prediction == 1 else "LIKELY LEGITIMATE",
        "confidence": f"{probability * 100:.1f}%",
        "flags": flags,
        "job_text": job_text[:300] + " ..." if len(job_text) > 300 else job_text
    }

    # Save to SQLite database
    save_analysis(job_text, result["verdict"], result["confidence"], flags)

    # Save to session history
    if "history" not in session:
        session["history"] = []

    session["history"].append({
        "verdict": result["verdict"],
        "confidence": result["confidence"],
        "preview": job_text[:100] + " ..." if len(job_text) > 100 else job_text,
        "flag_count": len(flags)
    })
    session.modified = True

    return render_template("result.html", result=result, app_name=APP_NAME)

@app.route("/history")
def history():
    analyses = get_all_analyses()
```

(Manages the submission of the form, executes the ML model and flag checks, applies the STRONG_FLAGS override if necessary, stores the outcome in the database, and provides the user with the decision.)

```
@app.route("/history")
def history():
    analyses = get_all_analyses()
    return render_template("history.html", analyses=analyses, app_name=APP_NAME)

if __name__ == "__main__":
    app.run(debug=True)
```

(Retrieves all past scan records from the database and renders them on the history page.)

(Manages the SQLite connection and defines functions to create the table, save each scan result, and retrieve past analyses.)

```
Session Actions Edit View Help
GNU nano 8.7.1 database.py
import sqlite3
from datetime import datetime

# Database file will be created in your Web_Tool folder
DATABASE = "database.db"

def get_db():
    # Connect to the database
    conn = sqlite3.connect(DATABASE)
    # This lets us access rows like dictionaries
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    # Create the database and tables if they don't exist yet
    conn = get_db()
    cursor = conn.cursor()

    # Create the analyses table
    # This stores every submission a user makes
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS analyses (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            job_text TEXT NOT NULL,
            verdict TEXT NOT NULL,
            confidence TEXT NOT NULL,
            flag_count INTEGER NOT NULL,
            flags TEXT NOT NULL,
            timestamp TEXT NOT NULL
        )
    """)

    conn.commit()
    conn.close()
    print("Database initialized successfully.")

def save_analysis(job_text, verdict, confidence, flags):
    # Save a single analysis to the database
    conn = get_db()
    cursor = conn.cursor()

    # Convert flags list to a readable string for storage
    flag_summary = " | ".join([f"flag" for f in flags]) if flags else "None"

    cursor.execute("""
        INSERT INTO analyses (job_text, verdict, confidence, flag_count, flags, timestamp)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (
        job_text,
        verdict,
```

```
        confidence,
        len(flags),
        flag_summary,
        datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    ))

    conn.commit()
    conn.close()

def get_all_analyses():
    # Retrieve all past analyses, most recent first
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT * FROM analyses ORDER BY timestamp DESC
    """)

    rows = cursor.fetchall()
    conn.close()
    return rows

def get_recent_analyses(limit=10):
    # Retrieve only the most recent analyses
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT * FROM analyses ORDER BY timestamp DESC LIMIT ?
    """, (limit,))

    rows = cursor.fetchall()
    conn.close()
    return rows
```

```
GNU nano 2.7.1 model/train_model.py
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
import pickle
import os

def train():
    # Load dataset
    df = pd.read_csv("data/fake_job_postings.csv")

    # Combine text columns into one
    df["text"] = (
        df["title"].fillna("") + " " + df["code"].fillna("") + " " + df["description"].fillna("") + " " + df["requirements"].fillna("")
    )

    # Input and label
    X = df["text"]
    y = df["fraudulent"]

    # Build pipeline
    pipeline = Pipeline([
        ("vectorizer", CountVectorizer(stop_words="english")),
        ("classifier", MultinomialNB())
    ])

    # Train the model
    pipeline.fit(X, y)

    # Save model to file
    os.makedirs("model", exist_ok=True)
    with open("model/model.pkl", "wb") as f:
        pickle.dump(pipeline, f)

    print("Model trained and saved successfully.")

if __name__ == "__main__":
    train()
```

```
(.venv)-(gina@kali)-[~/Web_Tool]
$ python3 model/train_model.py
Model trained and saved successfully.
```

(First iteration of train_model.py using Multinomial Naive Bayes and CountVectorizer, the starting point before the algorithm was upgraded.)

```
(.venv)gina@kali: ~/Web_Tool/model
Session Actions Edit View Help
GNU nano 2.7.1 train_model.py
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import pickle
import os

def train():
    # Load combined dataset of both original and newer job postings
    df = pd.read_csv("data/combined_job_postings.csv")

    X = df["text"]
    y = df["is_fake"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    pipeline = Pipeline([
        ("vectorizer", TfidfVectorizer(
            stop_words="english",
            max_features=5000,
            ngram_range=(1, 1)
        )),
        ("classifier", LogisticRegression(
            max_iter=1000,
            C=0.1,
            class_weight="balanced"
        ))
    ])

    pipeline.fit(X_train, y_train)

    predictions = pipeline.predict(X_test)
    print(classification_report(y_test, predictions))

    os.makedirs("model", exist_ok=True)
    with open("model/model.pkl", "wb") as f:
        pickle.dump(pipeline, f)

    print("Model trained and saved successfully.")

if __name__ == "__main__":
    train()
```

(Final Iteration, loads the combined dataset, builds the TF-IDF and Logistic Regression pipeline, trains the model, and saves it as model.pkl.)

(Contains the three CSV files used for training, the original EMSCAD dataset, the newer dataset, and the merged combined file.)

```
(.venv)-(gina@kali)-[~/Web_Tool/data]
$ ls
combined_job_postings.csv  fake_job_postings.csv  fake_real_job_postings_3000x25.csv
```

(templates/; index.html, result.html, and history.html)\

```
Session Actions Edit View Help
GNU nano 8.7.1 index.html
!DOCTYPE html>
<html>
<head>
  <title>{{ app_name }}</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <div class="container">
    <h1>{{ app_name }}</h1>
    <a href="/history" class="history-link"> View All Past Anaylses</a>
    <p>Paste a job posting below to check if it might be fraudulent. This tool will analyze the text and highlight any warning signs.</p>
    <form action="/analyze" method="POST">
      <textarea name="job_text" rows="12" placeholder="Paste job description here... "></textarea>
      <br>
      <button type="submit">Analyze Posting</button>
    </form>
    <p class="disclaimer">⚠️ This tool is meant to guide and educate, not provide a guaranteed verdict.</p>
  </div>
</body>
</html>
```

(Index.html, the main input page where users paste a job posting and submit it for analysis.)

```
Session Actions Edit View Help
GNU nano 8.7.1 history.html
[DOCTYPE html]
<html>
<head>
<title>History - {{ app_name }}</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="container">
<h1>{{ app_name }}</h1>
<h2>All Past Analyses</h2>

{% if analyses %}
<p>{{ analyses|length }} total submission(s) recorded.</p>
{% for item in analyses %}
<div class="history-card {{ 'history-fake' if 'FAKE' in item['verdict'] else 'history-legit' }}">
<div class="history-verdict">
{{ item['verdict'] }} - {{ item['confidence'] }} fraud probability
</div>
<div class="history-flags">
{{ item['flag_count'] }} warning flag(s): {{ item['flags'] }}
</div>
<div class="history-timestamp">
{{ item['timestamp'] }}
</div>
<div class="history-preview">
{{ item['job_text'][:150] }}...
</div>
</div>
{% endfor %}

{% else %}
<p>No analyses recorded yet. Go analyze a posting!</p>
{% endif %}

<a href="/">Back to Home</a>
</div>
</body>
</html>
```

(history.html, displays all stored scan records with the verdict, confidence score, flag count, timestamp, and a preview of the posting text.)

```
Session Actions Edit View Help
GNU nano 8.7.1 result.html
[DOCTYPE html]
<html>
<head>
<title>Results - {{ app_name }}</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="container">
<h1>{{ app_name }}</h1>
<h2>Analysis Result</h2>

<div class="verdict {{ 'fake' if 'FAKE' in result.verdict else 'legit' }}">
<h2>{{ result.verdict }}</h2>
<p>Fraud probability: <strong>{{ result.confidence }}</strong></p>
</div>

{% set confidence_val = result.confidence | replace('%', '') | float %}

{% if 'LEGITIMATE' in result.verdict and confidence_val >= 15 and confidence_val < 40 %}
<div class="caution-message">
<strong>Proceed with caution.</strong> While this posting appears legitimate,
our tool detected a low level of uncertainty. This could mean the posting contains
subtle characteristics sometimes associated with fraud. We recommend independently
verifying the company on LinkedIn, checking their official website, and never
sharing personal information until you have confirmed the employer is real.
</div>
{% endif %}

{% if 'FAKE' in result.verdict and confidence_val < 70 %}
<div class="caution-message">
<strong>Use your own judgment.</strong> Our tool flagged this posting but with
moderate confidence. It is possible this is a legitimate posting that contains
some unusual language. Cross reference the company name, contact details, and
job description independently before making any decisions.
</div>
{% endif %}

{% if result.flags %}
<div class="flags">
<h3>Warning Flags Detected ({{ result.flags|length }}):</h3>
{% for item in result.flags %}
<div class="flag-card">
<div class="flag-title">{{ item.flag }}</div>
<div class="flag-explanation">{{ item.explanation }}</div>
</div>
{% endfor %}
</div>
{% else %}
<div class="no-flags">
<p>No specific warning flags detected in this posting.</p>
</div>
{% endif %}

<div class="posting-preview">
<h3>Posting Preview:</h3>
<p>{{ result.job_text }}</p>
</div>

<a href="/">Analyze Another Posting</a>
</div>
</body>
</html>
```

```
</div>
{% else %}
<div class="no-flags">
<p>No specific warning flags detected in this posting.</p>
<p>While no flags were found, always research the company independently
before applying or sharing any personal information.</p>
</div>
{% endif %}

<div class="posting-preview">
<h3>Posting Preview:</h3>
<p>{{ result.job_text }}</p>
</div>

<a href="/">Analyze Another Posting</a>
</div>
</body>
</html>
```

(result.html, displays the analysis result including the verdict, fraud probability, any flags detected with explanations, and the no-flags reminder message.)

(Controls the visual layout of the web interface, including color coding for fake vs. legitimate verdicts, flag card styling, and the caution message design.)

```
GNU nano 8.7.1 style.css
body {
    font-family: "Times New Roman", Times, serif;
    background: #f4f4f4;
    margin: 0;
    padding: 0;
}

.container {
    max-width: 750px;
    margin: 40px auto;
    background: white;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

h1 {
    color: #2c3e50;
}

textarea {
    width: 100%;
    padding: 10px;
    font-size: 14px;
    border: 1px solid #ccc;
    border-radius: 5px;
}

button {
    margin-top: 10px;
    padding: 10px 20px;
    background: #2c7be5;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 15px;
}

button:hover {
    background: #1a5db5;
}

.verdict {
    padding: 20px;
    border-radius: 8px;
    margin: 20px 0;
    text-align: center;
}
```

```
GNU nano 8.7.1 style.css
.verdict {
    padding: 20px;
    border-radius: 8px;
    margin: 20px 0;
    text-align: center;
}

.fake {
    background: #ffe0e0;
    color: #c0392b;
}

.true {
    background: #e0ffe0;
    color: #27ae60;
}

.flags ul {
    list-style: none;
    padding: 0;
}

.flags li {
    background: #fff3cd;
    padding: 5px 12px;
    margin: 5px 0;
    border-left: 4px solid #f39c12;
    border-radius: 3px;
}

.posting-preview {
    background: #f9f9f9;
    padding: 15px;
    border-radius: 5px;
    margin-top: 20px;
    font-size: 13px;
    color: #555;
}

Disclaimer {
    font-size: 12px;
    color: #888;
    margin-top: 15px;
}

display: inline-block;
margin-top: 20px;
```

```
Session Actions Edit View Help
GNU nano 2.7.1 style.css
.discarder {
  font-size: 12px;
  color: #888;
  margin-top: 15px;
}
a {
  display: inline-block;
  margin-top: 20px;
  color: #2c7be5;
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
}
.history-timestamp {
  font-size: 11px;
  color: #999;
  margin-top: 4px;
}
.history-link {
  display: inline-block;
  margin-bottom: 20px;
  padding: 8px 15px;
  background: #2c3e50;
  color: white;
  border-radius: 5px;
  text-decoration: none;
  font-size: 14px;
}
.history-link:hover {
  background: #1a252f;
  text-decoration: none;
}
.caution-message {
  background: #fff8e1;
  border-left: 5px solid #f39c12;
  border-radius: 5px;
  padding: 15px;
  margin: 10px 0;
  font-size: 14px;
  color: #7d5a00;
  line-height: 1.7;
}
```

3.4b System Architecture Overview

The Client Layer consists of three HTML templates rendered by Flask. Students paste a job posting into index.html, results are displayed on result.html (verdict, confidence score, and flagged items with explanations), and history.html shows all past scans from the database.

The Server Layer is where the analysis runs. The Input Handler receives the submitted text, the Detection Engine applies the trained ML model to classify it, the Explanation Layer scans the text with regular expressions to identify specific red flags, and the Scan Logger saves each result to the database.

The Database Layer uses SQLite through Python's built-in sqlite3 library. On startup, init_db() creates the analyses table if it doesn't exist. Each scan is saved with the posting text, verdict, confidence score, flag count, flag summary, and timestamp. The /history route retrieves all records ordered by most recent.

3.4c Machine Learning Algorithms

The first iteration of the model was built using three components from Python's scikit-learn library. Pandas loaded the dataset and combined the job title, description, and requirements into one block of text per posting. CountVectorizer converted that text into word counts, and MultinomialNB learned which word patterns appeared more often in fake postings versus real ones. These steps were connected using a pipeline, so everything ran together as one process.

Naive Bayes worked as my main algorithm, but had a clear limitation: it treats every word as independent and equally weighted, which is not how language actually works. That led to switching to Logistic Regression with TF-IDF, which weights words more intelligently and handles the class imbalance between real and fake postings in the combined dataset.

Looking ahead, a Random Forest classifier would be a logical next step. It builds multiple decision trees and combines their results, making it more reliable in borderline cases, and it would work within the same scikit-learn setup without requiring a completely different implementation.

Dataset Evolution

The model was trained on the Kaggle EMSCAD dataset, but since the most relevant dataset was collected several years ago, some patterns didn't reflect how newer scam postings are written. A second, more recent dataset was merged in using pandas to improve coverage. The combined set had far more real postings than fake ones, so `class_weight="balanced"` was used to keep the model from defaulting toward always predicting legitimate.

3.4d Implementation

Development followed the benchmark phases outlined in the project proposal, starting with the Virajk19 open-source tool as the base. Once confirmed working in the local Kali environment, it was rebuilt and extended from there.

The detection engine was upgraded to Logistic Regression with TF-IDF, and the `WARNING_FLAGS` dictionary was expanded from four basic patterns to nine categories, each with a detailed explanation. The `check_flags()` function was built to handle rule-based checks for phone numbers, email formats, and known scam identifiers, with `STRONG_FLAGS` override logic added to adjust the verdict when a high-confidence red flag is detected.

Three HTML templates were created for the interface and styled with a custom CSS file. The result page uses Jinja2, a templating engine built into Flask that allows Python logic to run inside HTML, conditional logic to display caution messages when the model confidence is borderline, making sure users are never given a false sense of certainty in either direction.

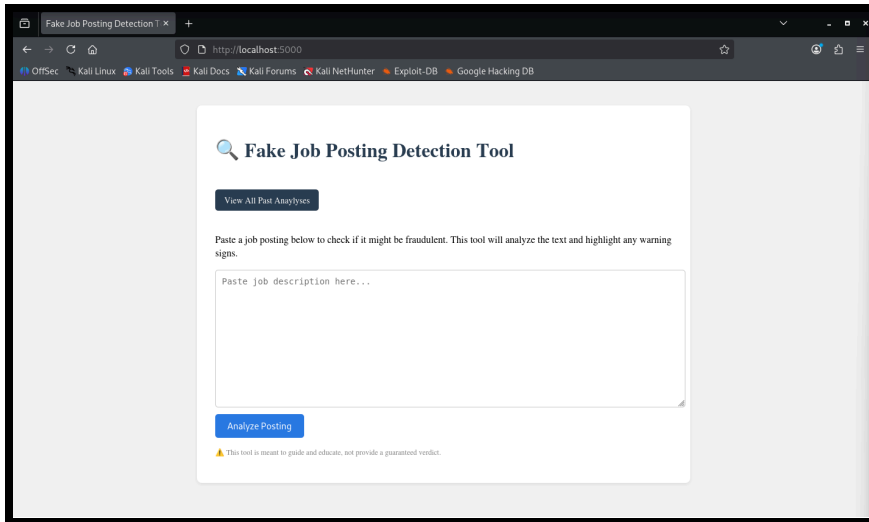
The SQLite database was connected through `database.py` to handle table creation, saving results, and retrieval. Flask session was also used to keep a temporary scan history for the current browser session. Testing was done using six postings ranging from obvious to deliberately tricky, which confirmed that the hybrid approach caught cases that neither the model nor the rule-based layer would have caught on their own.

3.5 Results

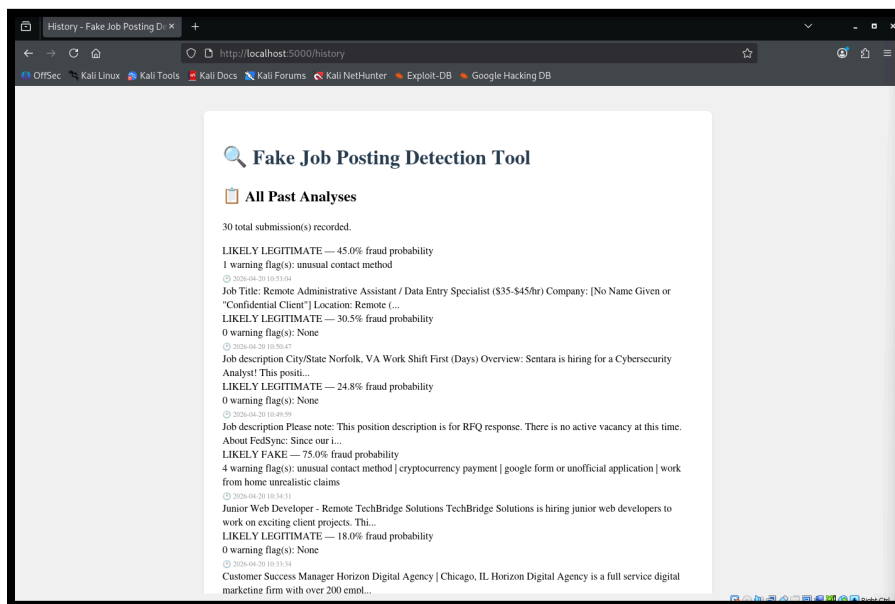
It's worth noting that this tool doesn't guarantee a definitive verdict. As no automated system can fully guarantee whether a posting is safe, users should always remember to verify the company independently before inputting any personal information. Overall, this upgraded tool gave a good attempt at getting to some accuracy, but without relevant yearly datasets for the model to run on, it can only train based on older models, which also reduces our accuracy on recent job postings now.

The tool was tested using six AI- Generated job postings, three postings from Handshake, and one WhatsApp posting. The generated postings consisted of two obvious fakes, two clearly legitimate, and two harder examples designed to test the tool to see how accurate it could get with flagging suspicious wording in the postings themselves. The obvious fakes were handled well, with knowledgeable flags indicating why the posting was seen as fake based on the model. The rule-based layer flagged warning patterns, including personal email addresses, Social Security number requests, telegram or WhatsApp contact methods, and guaranteed income claims, with most returning an explanation at the bottom highlighted in yellow.

The harder test showed both the strengths and limitations of the tool. Most postings with hidden red flags were still identified, though some required the rule-based layer to catch what the ML model had missed. There were also cases where the tool had returned a legitimate verdict on a posting that was clearly fake, specifically one generated specifically as a fake, because it did not contain any of the specific patterns the tool was trained to detect. This showed that the ML model relies mainly on the words and patterns it was trained on, so if a posting is written without common scam language, the tools can inaccurately detect the posting. That issue was exactly why the flag “ No specific warning flags detected in this posting.” While no flags were found, always research the company independently before applying or sharing any personal information.” This shows that even when the tool finds nothing suspicious, users are still reminded to research the company independently. The goal wasn't to guarantee completely accurate results; it was to provide a useful check for users who don't know what to look for to help them avoid scams or false postings.



(The tool's home page is running locally at `http://localhost:5000`, where users paste a job posting and submit it for analysis.)



(The `/history` page displays 30 recorded submissions, showing each verdict, fraud probability, flags detected, timestamp, and a preview of the posting text.)

 **Fake Job Posting Detection Tool**

Analysis Result

LIKELY FAKE
Fraud probability: 75.0%

 **Warning Flags Detected (5):**

 **requests personal info early**
Legitimate employers do not ask for sensitive personal information such as your Social Security Number, bank account details, or passport information during the application stage. This type of information is only needed after you have been officially hired and is handled through secure HR systems. If a job posting or recruiter asks for this upfront, it is a strong indicator that they are attempting to collect your data for identity theft or financial fraud. Never share this information before verifying the employer is real.

 **unusual contact method**
Professional companies communicate through official business email addresses or phone systems, not personal messaging apps like WhatsApp or Telegram. When a recruiter insists on communicating through these platforms it is often because they want to avoid a paper trail and make it harder for you to verify their identity. Scammers use these apps because accounts are easy to create, hard to trace, and allow them to impersonate real companies. Always insist on communicating through official company channels.

 **remote only interview red flag**
While remote interviews are common and legitimate, postings that emphasize you will be hired without any real interview process or that explicitly avoid any in person or video verification are suspicious. Scammers avoid face to face contact because it makes them easier to identify and report. A legitimate employer will always want to properly vet candidates through a real interview process.

(A test result based on one of the generated sample postings, returning **LIKELY FAKE** at 75.0% fraud probability, with five warning flags detected, including requests for personal information early, unusual contact method, and a remote-only interview red flag.)

 **Fake Job Posting Detection Tool**

Analysis Result

LIKELY LEGITIMATE
Fraud probability: 18.0%

 **Proceed with caution.** While this posting appears legitimate, our tool detected a low level of uncertainty. This could mean the posting contains subtle characteristics sometimes associated with fraud. We recommend independently verifying the company on LinkedIn, checking their official website, and never sharing personal information until you have confirmed the employer is real.

 **No specific warning flags detected in this posting.**

While no flags were found, always research the company independently before applying or sharing any personal information.

Posting Preview:

Customer Success Manager Horizon Digital Agency | Chicago, IL. Horizon Digital Agency is a full service digital marketing firm with over 200 employees across 3 offices in the US. We are looking for a Customer Success Manager to join our Chicago team.
Responsibilities: - Manage a portfolio...

[← Analyze Another Posting](#)

(A result returning **LIKELY LEGITIMATE** at 18.0% fraud probability with no flags detected, the caution message still appears, reminding the user to independently verify the company before sharing any personal information, demonstrating the tool's built-in awareness feature.)

3.6 Conclusion

This project produced a working web-based fake job posting detection tool that combines machine learning classification with rule-based pattern detection and plain-language explanations. Starting from an open-source tool from GitHub, the classification algorithm was upgraded from Multinomial Naive Bayes to Logistic Regression with TF-IDF. The dataset was expanded to get recent data that could better reflect how scam postings are written today, and an explanation layer was added to make the tool educational rather than just functional.

The biggest takeaway was that a hybrid approach is more reliable than relying on one method alone. The ML model handles general pattern recognition, but the rule-based layer catches specific identifiers it would miss, and together they work better than either could on its own.

Working with the training data also taught me a lot. The first iteration struggled against newer posting styles, and merging in a more recent dataset made a clear difference. It also highlighted why class imbalance matters. When the training data has far more real postings than fake ones, the model has to be told not to treat that as the default answer.

Looking ahead, future improvements could include expanding the scam blocklist, adding URL scraping so users can submit a link instead of pasting text, and building in a feedback mechanism so incorrect results can be flagged and used to improve the model over time. Deploying the tool to a cloud environment would also make it accessible to students without any local setup required.

References

- Bansal, S. (2018). *Real or fake? Fake job posting prediction [Dataset]*. Kaggle.
<https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction>
- GeeksforGeeks. (n.d.). *Naive Bayes classifiers*.
<https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>
- Pallets Projects. (n.d.). *Flask documentation (stable)*.
<https://flask.palletsprojects.com/en/stable/>
- Pillai, A. S. (2023). *Detecting fake job postings using bidirectional LSTM*. *International Research Journal of Modernization in Engineering Technology and Science*, 5(3).
<https://www.doi.org/10.56726/IRJMETs35202>
- scikit-learn developers. (n.d.). *sklearn.ensemble.RandomForestClassifier*. scikit-learn.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Virajk19. (2025). *fake-job-detector [Source code]*. GitHub.
<https://github.com/Virajk19/fake-job-detector>
- Yadav, K. (2025). *Fake vs real job postings (synthetic NLP dataset) [Dataset]*. Kaggle.
<https://www.kaggle.com/datasets/khushikyad001/fake-vs-real-job-postings-synthetic-nlp-dataset>

